

OpenCollab SWE-bench V3 评测 Harness 说明与审核报告

Kai / Codex

2026-07-06

1 目标

OpenCollab 的 SWE-bench 做题框架把四件事合并管理：生成 patch、保存可恢复现场、发现可评测 patch、启动 official eval。V3 类 workflow 的结构化阶段多、单题时间长、远端 Docker 与模型代理都可能抖动，因此评测 Harness 的第一目标是把基础设施错误、空 patch、语义失败和官方评测结果拆开，避免一个可恢复的 transport failure 被写成模型能力失败。

当前 Harness 的基本原则是：模型生成阶段产生 patch；checkpoint 在语义状态之外保存容器工作区 diff；watchdog 根据状态决定继续生成、暂停等待、恢复现场或启动 official eval；official eval 完成后再把 resolved/unresolved 写回状态文件。任何真实 API key、临时代理 token 或上游 token 都不能进入日志、Markdown 或远端持久文件。

2 整体结构

层级	代表文件	职责
Workflow runtime	opencollab/opencollab/application/workflow.py	管理结构化 workflow, transport retry、failure counts、checkpoint 读写、worktree sidecar 捕获与恢复。
Eval harness	opencollab/opencollab/harness/evaluator.py	创建容器环境, 注入 SWE-bench 测试, 接入 checkpoint 恢复, 捕获 budget、infra、timeout 和异常, 返回可观测 EvalResult。
Patch 生成器	swebench/gen_prediction_workflow.py	自动单题 workflow, 抽取容器 diff, 清理验证产物, 空 patch 时从 checkpoint sidecar 抢救, 写 predictions 与 metrics。
并发启动器	scripts/swe_parallel_start.py	按任务拆分独立 run 目录和 tmux session, 支持 --resume, 控制远端代理模式、预算、超时和数据集。
做题 watchdog	scripts/swe_v3_wave_watchdog.py	汇总多个 run, 分类任务状态, 修复远端代理, 触发 resume, 发现可评测 patch 后启动 official eval。
官方评测驱动	scripts/swe_auto_eval_driver.py	按 record_id 和 patch sha 创建 eval side-car, 规范化 dataset, 启动 official eval, 发现 report, 写 status。
远端执行脚本	scripts/run_swe_v2_one_from_remote.py	在远端 tmux 内执行单题生成, 接收受控 credential, 设置代理环境, 追加 generation log。

3 状态机

watchdog 的状态分类是做题框架的核心。一个任务先从 no_output 或 partial_output 进入生成; 生成中为 generating; 若 patch 非空且 workflow done, 则进入 patch_done 并等待 official eval; 若 official eval 完成, 则进入 official_resolved 或 official_unresolved。可恢复错误进入 infra_paused; 已有 checkpoint sidecar 的中断任务进入 recoverable_checkpoint; 空 patch 只在没有可恢复 sidecar 时进入 empty_patch_invalid。

状态	触发条件	下一步
generating	tmux session 仍活跃	只读监控, 不启动 official eval。
patch_done	patch 非空且 workflow_status=done	交给 swe_auto_eval_driver.py 启动 official eval。

<code>patch_done_advisory_gap</code>	patch 非空且 <code>workflow_status=advisory_gap</code>	默认作为 <code>diagnostic_only</code> 候选保留; 只有显式打开 <code>--eval-advisory-gap</code> 时才送 official eval。
<code>infra_paused</code>	transport、429、远端代理、 Docker、tmux、SSH 等可恢 复错误	等待健康恢复和冷却时间，随后 <code>--resume</code> 。
<code>recoverable_checkpoint</code>	任务 inactive，patch 为空， 但 checkpoint sidecar 存在	新容器恢复 <code>checkpoint.worktree.patch</code> 后继 续 workflow。
<code>empty_patch_invalid</code>	已写 prediction，patch 为 空，且没有可用 sidecar	保留日志，进入人工诊断。
<code>official_unresolved</code>	official eval 完成且 patch 未 解决	进入语义分析和下一轮算法修正。
<code>official_resolved</code>	official eval 完成且 patch 解 决	计为通过样本。

当前 summary 结束条件也按这个状态机调整。`infra_paused` 和 `recoverable_checkpoint` 不计入完成数；official eval 完成、终结性 infra invalid、诊断候选和没有可恢复 sidecar 的 empty patch invalid 进入最终 summary。generation technical failure 保留为待修复状态，修好脚本、镜像、dataset 或远端环境后继续推进。

4 结构化调用与 transport 错误

V2 的主要污染来自结构化 stage 失败。旧逻辑中，Connection error 可能被转成 None，再被 workflow fallback 解释成 verifier fail。当前 runtime 将 transport failure 单列，并给 schema-bound stage 加同 label 重试。Connection error、timeout、HTTP 408/409/429、5xx、proxy、tunnel、remote protocol 等都归入 transport 类；429 单独标为 `rate_limit_error`。重试耗尽后抛出 `WorkflowInfraError`，由 evaluator 转成带 `retryable_infra` 的 workflow result。

错误类型	当前分类	影响
Connection error / timeout	transport_error	原地重试，耗尽后进入 retryable infra。
HTTP 429	rate_limit_error	保留 pause reason，等待冷却后 resume。
schema capture 缺失	schema_failure_count	不混入 transport 统计。
语义 blocker	semantic_blocker_count	只有模型成功返回 verdict 后才进入 语义状态。
认证、权限、上下文长度	non-retryable	记录为配置或输入问题，避免盲目重 试。

5 Checkpoint 与 worktree sidecar

checkpoint 现在保存两类信息。第一类是语义状态，位于 `checkpoint.json`，包括 `workflow stage`、结构化产物、`token offset` 和 `runtime failure counts`。第二类是容器工作区 `diff`，位于 `checkpoint.worktree.patch`，配套元数据位于 `checkpoint.worktree.json`。`diff` 捕获使用临时 `git index` 执行 `git add -A` 和 `git diff --cached --binary`，并排除 SWE-bench 注入测试文件，避免把官方测试 `patch` 作为模型修改保存。

恢复时，`evaluator` 在新容器创建后立即调用 `restore_checkpoint_worktree()`。若当前 `worktree` 为空，则应用 `sidecar patch`；若当前 `worktree` 已经有 `diff`，则跳过 `apply`，保护已有现场。若 `sidecar patch` 无法应用，`metrics` 写入 `checkpoint_sidecar_unusable`，任务保留为诊断对象，不能提交文本 `fallback` 或空 `patch`。

机制	作用
<code>checkpoint.worktree.patch</code>	保存模型已经写入容器但尚未 official eval 的源码 <code>diff</code> 。
<code>checkpoint.worktree.json</code>	保存 <code>patch sha256</code> 、字节数、 <code>stage</code> 、 <code>HEAD</code> 、 <code>status short</code> 、 <code>preserved_previous_patch</code> 和 <code>submission_eligible</code> 。
空 <code>diff</code> 与 <code>capture failure</code> 保留旧 <code>sidecar</code>	新 <code>checkpoint</code> 捕获为空或捕获命令失败时，保留已有可校验 <code>sidecar</code> ，避免空 <code>patch</code> 或临时基础设施错误覆盖非空 <code>patch</code> ；这类 <code>sidecar</code> 的 <code>submission_eligible=false</code> ，只用于恢复现场。
恢复前探测 <code>current diff</code>	当前容器已有修改时跳过 <code>sidecar apply</code> ，避免重复应用。
<code>runtime</code> 异常 <code>checkpoint</code>	<code>budget</code> 、 <code>infra</code> 、 <code>timeout</code> 和普通异常都会触发 <code>_runtime checkpoint</code> ，尽量保留当时 <code>diff</code> 。

6 这次审核发现与修复

这次独立审核由两个只读 subagent 完成。Chandrasekhar 检查 V3.1 workflow 协议，指出 ETV 状态枚举、final-gate fallback、coverage gate 和重新仲裁分支的风险。Descartes 检查评测 Harness，指出 checkpoint sidecar、ready eval 候选身份、technical eval failure 回读和 report 技术失败分类的风险。两路审查共同指向同一个结论：workflow checkpoint 与 sidecar 的方向正确，official eval 的 hash side-car 与 tmux/flock 设计也可用；需要继续补厚的是 watchdog 和评测 driver 对半写、错配和可恢复异常的处理。

第一个问题是 `checkpoint_summary()` 过去依赖 `checkpoint.json` 能被成功解析。一旦这个 JSON 在 NFS 或进程中断下损坏，旁边仍然存在的 `checkpoint.worktree.patch` 会被忽略。随后 watchdog 可能把任务分到 fresh start, `swe_parallel_start.py` 会归档 `trajectory` 目录，sidecar 也随之被移走。修复后，watchdog 会独立检查 `checkpoint.worktree.patch` 和 `checkpoint.worktree.json`；即使 `checkpoint.json` 损坏，只要 sidecar patch 非空，就标为 `checkpoint_reusable`，下一次启动强制走 `--resume`。

第二个问题是 `predictions.jsonl` 与 `metrics.jsonl` 过去分别 append，进程若在两次写入之间退出，watchdog 可能把新 prediction 与旧 metrics 配成一对。修复后，生成器为每次产出写入同一个 `record_id` 和 `patch_sha256`；watchdog 与并发启动器只接受同 `record_id` 的成对记录。若发现 prediction 已写而对应 metrics 尚未落盘，任务进入 `partial_output` 并等待下一轮或 resume。watchdog 启动 `swe_auto_eval_driver.py` 时传入当前 `record_id` 与 `patch_sha`，official eval driver 只读取这个候选 patch，避免历史 patch 被误评。

第三个问题是一次新的空 patch 可能遮住旧的非空 patch。Pro Lite 十题里出现过“前面生成过 diff，续跑后新容器干净，最终写出空 patch”的症状。修复后，最新 prediction/metrics 记录控制当前任务状态；旧的非空且 workflow done patch 只写入 `last_good_patch_*` 字段，作为人工抢救和恢复线索，不能替代当前行触发 official eval。runtime 侧也同步加厚：空 diff 捕获时若已有非空 `checkpoint.worktree.patch`，metadata 写为 `empty_worktree_preserved_previous_patch`；capture failure 时若旧 sidecar 仍可校验，metadata 写为 `failed_preserved_previous_patch`。这两类保留 sidecar 的 `submission_eligible=false`，恢复时可用，最终提交时不会直接变成 prediction。

第四個问题是 summary 与 status JSON 过去直接 `write_text`。如果进程在写文件中途退出，下一轮 `load_json()` 会读到半截 JSON 并返回空状态。修复后，watchdog 的 status、final summary，official eval 的 state、summary、status、输入 prediction 和 dataset 都改成临时文件写入后 `os.replace` 原子替换。中断时保留旧状态，临时文件可以被后续清理。

第五个问题是结束条件过早。旧的 `_done_enough()` 把 `generation_technical_failed` 和 `technical_eval_failed` 算作完成，official eval watch 也把缺 dataset、缺依赖、缺镜像这类 blocked 状态算作 idle。修复后，可继续的技术状态不会触发最终 summary；watchdog 下一轮仍能修脚本、补镜像、恢复代理或重启 eval。

7 自动 official eval

official eval 是做题完成的判定器。生成 patch 后, watchdog 只有在 patch 非空、workflow done、生成 tmux inactive、同题 eval 未完成且未活跃时,才把任务列入 `ready_for_eval`。若结果带 `advisory_gap` 或 `done_with_advisory_gap`, 默认写成 `diagnostic_only`; 显式打开 `--eval-advisory-gap` 才按诊断模式送评。eval driver 根据 run 目录、instance id、record_id 和 patch sha 创建唯一 side-car, 避免同一题历史 patch 与新 patch 混用。

`swe_auto_eval_driver.py` 做三件事: 发现 candidate patch、启动官方评测、读取 report。它支持 SWE-bench Lite 和 Swe Batch Pro Lite 数据集发现; 遇到单对象 JSON 会规范化成单元素 dataset; report 查找覆盖 side-car 内 report、官方 harness 嵌套 report 和 `/home/dongyh` 下的全局 report。driver 现在按 record_id 和完整 patch sha 过滤候选, 并反查同一 record_id 且同一 patch hash 的 metrics; 没有 hash 一致的 eligible metrics, candidate 会写成 `blocked_missing_metric` 或 `blocked_metric_not_eval_eligible`, 不会启动 official eval。report 中的 patch apply 失败、error ids、empty patch ids、缺 report、缺镜像、缺 dataset、缺依赖都写成技术评测状态, 并与 official unresolved 分开。

8 远端代理与 credential

模型调用支持三种 credential 模式。常用模式是 `proxy-token` 或 `proxy-open`, 远端通过 SSH 反向隧道访问本机代理。短期离线需求可以使用 `encrypted-upstream-env`, 远端脚本用临时密钥解密 env, 启动本地代理后删除明文 env 文件。无论哪种模式, 日志脱敏和状态文件都不能输出真实 token。

远端代理 health 只能证明端口活着, 无法证明每次上游调用都稳定。因此 Harness 不能把 health OK 当成结构化 stage 一定成功的证据。当前设计把代理 health 用于是否修复隧道、是否解除 `infra_paused` 的条件之一; 实际结构化调用仍由 runtime 记录 transport failure、rate limit failure 和 retry attempts。

9 测试与自检

当前已有针对性测试覆盖了几类重要边界。这次使用 `py_compile` 覆盖改动脚本, 并在 `opencollab/.venv` 中执行 `pytest`, 目标文件 83 个测试全部通过。新增测试覆盖了损坏 checkpoint JSON 仍识别 sidecar、record_id+patch hash 双重绑定、最新空 patch 不会被旧非空 patch 替代、`last_good_patch_*` 仍保留抢救线索、blocked eval 等待修复、advisory gap 默认诊断、preserved sidecar 禁止最终提交、空 diff 和 capture failure 显式保留上一份可恢复 patch。

测试文件	覆盖内容
opencollab/tests/test_workflow_token_offset.py	checkpoint context offset、diff command 注入路径排除、rate limit 分类。
opencollab/tests/test_sidecar_recovery.py	checkpoint 恢复时当前 workflow 非空跳过、sidecar apply 失败时禁止提交 fallback、preserved sidecar 只可恢复不可提交。
opencollab/tests/test_sidecar_health.py	进入 resume group, retryable infra 进入暂停, Docker daemon failure 暂停, auto-eval 需要 eligible metrics。
opencollab/tests/test_validator_infra_error.py	Validator Infra Error 被转换成 infra_invalid result, checkpoint 写入 runtime 状态。

10 当前保障边界

当前 Harness 可以保障 checkpoint 边界之后的恢复。也就是说，只要 workflow 已经成功写过 checkpoint，语义状态、token offset 和容器 diff 都有机会在新容器里恢复。它也可以保障大多数可恢复基础设施错误进入暂停态，等待代理、Docker、SSH 或 tmux 恢复后继续。

当前 Harness 仍然无法覆盖任意时刻的物理中断。若进程在一次 LLM 调用未返回时被杀死，或者容器在 checkpoint 写入之前消失，这段调用消耗和未落盘修改仍可能丢失。若 sidecar patch 与新容器 base 不兼容，恢复会进入 `checkpoint_sidecar_unusable`，需要人工诊断。若 official eval 依赖的镜像或 dataset 缺失，自动脚本会阻塞并保留证据，但不会把 patch 判成 resolved 或 unresolved。

11 独立审核结论

两路审核的共同结论可以概括为：V3 失败暴露出的核心风险已经从 workflow 语义转移到 harness 可靠性。经过这次补厚，sidecar patch 在 checkpoint JSON 损坏、空 diff 捕获、capture failure 时仍能被发现或显式保留；prediction 与 metrics 有稳定配对键；official eval 只评当前 `record_id/patch sha`；状态与 summary JSON 采用原子替换；可恢复技术状态停在等待修复的状态，而非提前进入最终 summary。

这意味着之后再遇到模型代理断连、远端 Docker 异常、tmux 中断、official eval 缺镜像或 dataset 这类问题，自动脚本会尽量暂停并保留证据。修复环境后，watchdog 可以根据 checkpoint 和 sidecar 继续推进。若没有 checkpoint 或 sidecar，脚本会把任务保留为诊断对象，并避免把这种技术中断解释成 V3 算法失败。

12 评审迭代门槛

这次经验说明，复杂 workflow 和评测 Harness 不能以一次自检作为终点。之后所有 V3.1、V3.2 或做题脚本的结论都采用分级表述：一次本地 smoke 通过只表示语法和样例路径可运行；一次独立审查通过只表示该轮没有新增 P0/P1；只有连续两轮独立审查都没有新增 P0/P1，并且真实恢复 smoke 能

从中断点继续生成非空 patch、按当前 `record_id/patch sha` 启动 official eval，才允许把该版本标为可进入小样本实验。

评审也要覆盖不同角度。一个审查者看 workflow 协议，重点检查 advisory gap、coverage gate、ETV failure、final gate fallback 是否被正确分类；另一个审查者看 Harness，重点检查 checkpoint sidecar、prediction/metrics 配对、状态 JSON 原子写、official eval report 分类和 retry/pause 语义。任何审查者提出 P0/P1 后，都要先修代码和测试，再重新编译报告。没有新 P0/P1 时，也要保留剩余风险，例如 LLM 调用未返回前的物理中断、sidecar 与新容器 base 不兼容、远端镜像缺失这类边界。

13 下一步建议

下一步应做一个真实恢复 smoke。选择 `django__django-11019` 或 Pro Lite 中已有 sidecar patch 的样本，启动 V3 生成后在 checkpoint sidecar 出现时人为断开远端代理或杀掉 tmux，让 watchdog 进入 `infra_paused` 或 `recoverable_checkpoint`。恢复代理后运行同一 watchdog，确认启动命令带 `--resume`，确认新容器日志出现 `checkpoint worktree restored`，确认最终 patch 非空并自动进入 official eval。

这个 smoke 通过后，V3.1 还应减少硬结构化 stage 数量。candidate contracts、post-validation factory、judge triage 这类辅助 stage 应改成 advisory evidence producer，连接失败时复用缓存或跳过；最终法庭保留为少数硬裁决。这样可以降低 `APIConnectionError` 对整题完成率的乘法伤害。